ENTREGÁVEL DA SEMANA 9 — ARQUITETURA DA APLICAÇÃO

Projeto Final: Sistema de Apoio ao 1º Hackathon

Engenharia de Software I — IFPR Campus Pinhais — Profa. Lauriana Paludo

Como preencher: as linhas em amarelo são exemplos (apague-as ou substitua pelos dados reais da sua equipe). Preencha as linhas em branco e os campos vazios; acrescente linhas se precisar. Consulte o "Exemplo Resolvido" (projeto fictício) para ver o nível de detalhe esperado.

1. Identificação da equipe

Nome da equipe	HackaToon - Ideias Malucas. Código de Verdade
Integrantes (nome completo)	Daniel de Sousa Vetterlein Davi Takashi Bueno Umeda Victor Luiz Ostrovski Vitor Joaquim da Cruz Rodrigues
Link do repositório / GitHub Projects	https://github.com/davitbumeda/HackaToon
Data da entrega	23/10/2026

2. Requisitos não-funcionais que guiam a arquitetura

Mínimo de 3. Cada requisito deve ter meta mensurável e indicar como ele influencia a arquitetura.

ID	Requisito não-funcional	Meta mensurável	Como influencia a arquitetura	Histórias afetadas
RNF-01	*Desempenho do painel de classificação*	*Carregar em até 2 s com 200 equipes simultâneas*	*Consulta de classificação otimizada e resposta pronta para leitura*	*US-05*
RNF-02	Segurança (Bloqueio de Login)	Após 3 tentativas de senha incorretas consecutivas, o sistema deve bloquear o login por 15 minutos e enviar um e-mail de alerta ao usuário.	Exige que o Supabase controle o estado de tentativas no Banco de Dados e integre com um serviço externo de envio de e-mails.	*Histórias de Login*
RNF-03	Usabilidade (Responsividade)	O layout da aplicação web deve se adaptar e funcionar sem quebras visuais em telas de smartphones (a partir de 320px de largura).	O Front-end deve ser construído com design responsivo (CSS Media Queries, Flexbox) consumindo o BaaS.	Todas as histórias visuais

ID	Requisito não-funcional	Meta mensurável	Como influencia a arquitetura	Histórias afetadas
RNF-04	Segurança (Criptografia)	100% das senhas cadastradas devem ser salvas no Banco de Dados usando algoritmo de hash, nunca em texto limpo.	A camada de autenticação (Supabase Auth) precisa garantir o hash criptográfico nativo antes de salvar.	Histórias de Cadastro
RNF-05	Desempenho (Tempo de Resposta)	A tela inicial e o painel público de classificação devem carregar completamente em até 2 segundos em uma conexão de banda larga.	Exige que o Front-end seja leve (hospedagem estática no GitHub Pages) e consuma dados otimizados via Supabase Client.	Tela Inicial e Painel

3. Diagramas de arquitetura

Anexe as imagens dos diagramas (draw.io, Astah, Figma ou desenho à mão fotografado) ou informe o link. Depois descreva cada contêiner na tabela.

Diagrama de contexto (C4 nível 1) — imagem ou link	
Diagrama de contêineres (C4 nível 2) — imagem ou link	

Contêiner	Tecnologia	Responsabilidade	Comunica-se com (e como)
API REST (back-end)	*Node.js + Express*	*Regras do Hackathon, autenticação e acesso aos dados*	*Banco de dados (SQL); Aplicação web (HTTPS/JSON)*
Aplicação Web (Front-end)	HTML5, CSS3, JavaScript	Interface responsiva, formulários de inscrição, landing page e painel de resultados.	Supabase (HTTPS/JSON via Supabase Client)
Supabase (BaaS e Banco de Dados)	PostgreSQL + Supabase Auth	Autenticação de usuários, regras de segurança (RLS), armazenamento de dados e disparo de e-mails.	Aplicação Web (HTTPS/JSON); Serviço de E-mail externo (SMTP)

4. Estilo arquitetural e organização do código

Estilo adotado (monolito em camadas, microsserviços ou outro)	Arquitetura Serverless / Backend-as-a-Service (BaaS) com Front-end estático desacoplado.
Por que esse estilo atende aos requisitos não-funcionais	A adoção de um BaaS (Supabase) atende perfeitamente à necessidade de encriptação segura (RNF-03) e controle de tentativas falhadas (RNF-01) nativamente. A separação do front-end estático garante alto desempenho e carregamento rápido (RNF-04).
Organização do código (pastas, módulos ou camadas)	Foco no Front-end. Pastas: /pages (HTML para inscrição/painel), /styles (CSS responsivo), /scripts (lógica JS e integração com o Supabase Client) e /assets (imagens/ícones).
Tecnologias, ferramentas e como será a publicação (deploy) com orçamento zero	Front-end hospedado de forma estática no GitHub Pages (gratuito). Back-end em Node.js e Banco de Dados MySQL hospedados no plano gratuito/starter da plataforma Railway, com controle de código versionado no GitHub.

5. Registro de decisão arquitetural (ADR)

Título da decisão	ADR-001. Adoção de arquitetura Backend-as-a-Service (BaaS) com Supabase
Contexto (problema ou requisito que motivou a decisão)	Necessidade de desenvolver o sistema com orçamento zero, prazo restrito e garantia de segurança (RNFs 01 e 03), com uma equipe organizadora que possui pouco tempo disponível.
Decisão (o que a equipe escolheu)	Adotar o Supabase (PostgreSQL + Auth) como Backend-as-a-Service (BaaS) aliado a um Front-end estático (GitHub Pages), substituindo o desenvolvimento de uma API REST do zero.
Alternativas consideradas e por que foram descartadas	1) API REST em Node.js com MySQL: descartada pelo tempo de desenvolvimento. 2) Microsserviços: descartada por alta complexidade desnecessária para o escopo.
Consequências (o que se ganha e o que se aceita como custo ou risco)	Ganhos: Agilidade, segurança nativa (Auth) e custo zero. Riscos: Dependência de serviço terceirizado (vendor lock-in) e limites do plano gratuito.

6. Ligação com o backlog

Mostre que a arquitetura atende às histórias Must Have do backlog da Semana 8.

História (ID)	Parte da arquitetura que a atende
US-01	*Aplicação web + API REST + banco de dados*

7. Checklist de entrega

Marque (substitua ☐ por ☑) apenas o que a equipe realmente concluiu.

[x] Pelo menos 3 requisitos não-funcionais, cada um com meta mensurável

[ ] Diagrama de contexto (C4 nível 1) e de contêineres (C4 nível 2) anexados ou linkados

[x] Cada contêiner tem tecnologia, responsabilidade e comunicação indicadas

[x] Estilo arquitetural e organização do código descritos

[x] ADR com contexto, decisão, alternativas consideradas e consequências

[ ] Cada história Must Have do backlog está ligada a uma parte da arquitetura